Simulation Study - Validate the Pseudo-value Regression under the Null Setting

FengQi (Cynthia) Song

2026-03-03

Contents

1 Objective **1**

1.1 Sanity check 1

1.2 Monte Carlo validation 2

1.3 Sensitivity analysis(different evaluation time) 3

1.4 Pseudo-value Regression Model 5

1.5 Type I error 6

---

1 Objective

1.1 Sanity check

```
library(MCC)
source("01_generate_data.R")
t_eval <- 5
pseudo <- MCC::GenPseudo(data = dat, tau = t_eval)

str(pseudo)
```

```
## 'data.frame':    100 obs. of  3 variables:
##  $ idx   : num  1 2 3 4 5 6 7 8 9 10 ...
##  $ psi   : num  0.0299 1.1538 1.3798 8.5717 -2.2361 ...
##  $ pseudo: num  5.47 6.59 6.82 14.01 3.2 ...
```

```
head(pseudo)
```

```
##   idx         psi    pseudo
## 1   1  0.02993847  5.465204
## 2   2  1.15377624  6.589041
```

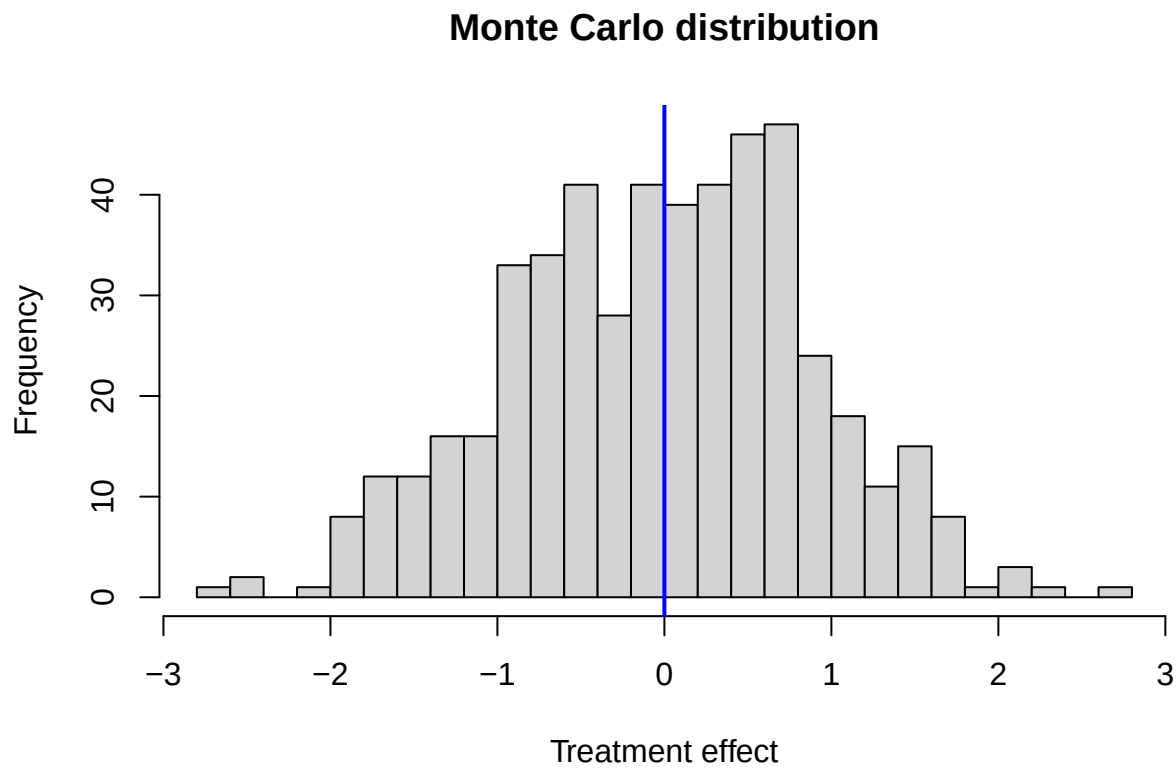
```
## 3   3  1.37984146  6.815107
## 4   4  8.57167973 14.006945
## 5   5 -2.23608779  3.199177
## 6   6  2.68130965  8.116575
```

```r
AX <- unique(dat[, c("idx", "A", "X")])
merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

tapply(merged$pseudo, merged$A, mean)
```

```
##        0        1
## 5.275612 5.594918
```

1.2 Monte Carlo validation

```r
library(MCC)
source("01_generate_data.R")

set.seed(123)

B <- 500
delta_hat <- numeric(B)

for (b in 1:B) {
  source("01_generate_data.R")
  pseudo <- MCC::GenPseudo(data = dat, tau = 5)

  AX <- unique(dat[, c("idx", "A")])
  merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

  delta_hat[b] <- mean(merged$pseudo[merged$A == 1]) -
                  mean(merged$pseudo[merged$A == 0])
}

hist(delta_hat, breaks = 30,
     main = "Monte Carlo distribution",
     xlab = "Treatment effect")
abline(v = 0, col = "blue", lwd = 2)
```



This distribution is centered at zero, suggesting no bias

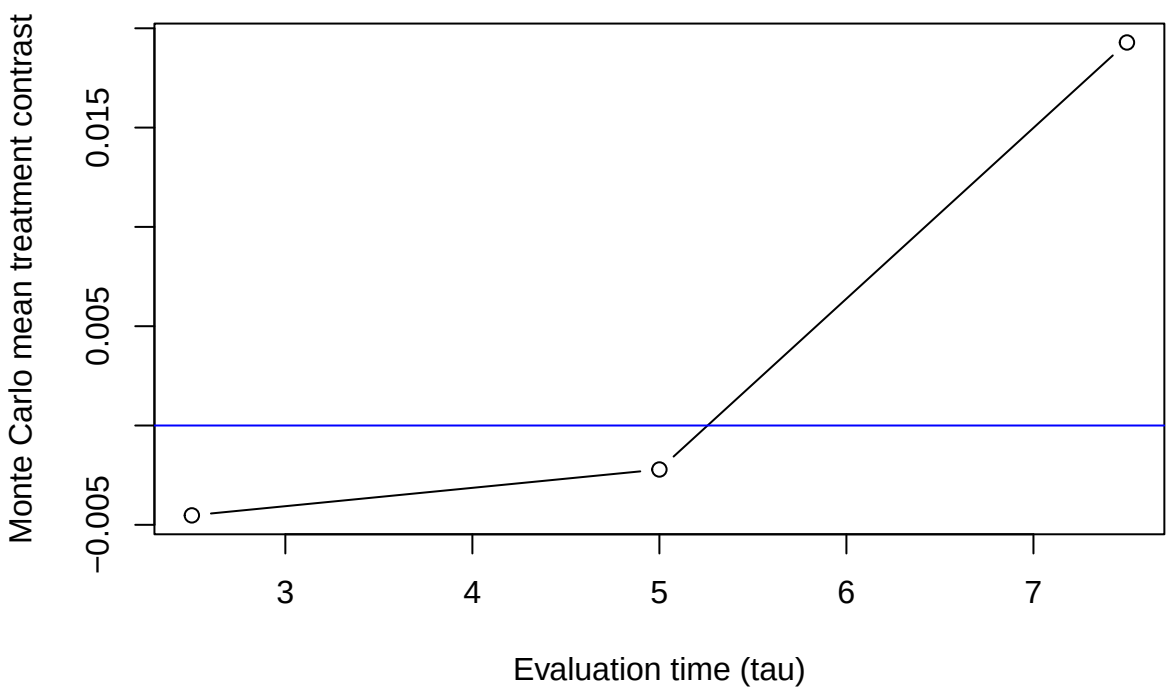
1.3 Sensitivity analysis(different evaluation time)

```
set.seed(123)

B <- 500
t_list <- c(2.5, 5, 7.5)

out_time <- data.frame(
  t_eval = t_list,
  mean_diff = NA_real_,
  sd_diff = NA_real_
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
  delta_tau <- numeric(B)

  for (b in 1:B) {

    source("01_generate_data.R")
```

```r
    pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    merged_j <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

    delta_tau[b] <- mean(merged_j$pseudo[merged_j$A == 1]) -
                    mean(merged_j$pseudo[merged_j$A == 0])
  }

  out_time$mean_diff[j] <- mean(delta_tau)
  out_time$sd_diff[j]   <- sd(delta_tau)
}

out_time
```

```
##   t_eval    mean_diff   sd_diff
## 1    2.5 -0.004524125 0.3088612
## 2    5.0 -0.002214517 0.8586752
## 3    7.5  0.019285168 1.7620563
```

```r
plot(out_time$t_eval, out_time$mean_diff, type = "b",
     xlab = "Evaluation time (tau)",
     ylab = "Monte Carlo mean treatment contrast")
abline(h = 0, col = "blue")
```



1.4 Pseudo-value Regression Model

```r
# Single-run regression across tau
t_list <- c(2.5, 5, 7.5)
AX <- unique(dat[, c("idx", "A")])

reg_out <- data.frame(
  t_eval = t_list,
  beta_A = NA,
  p_A = NA
)

for (j in seq_along(t_list)) {
  pseudo_j <- MCC::GenPseudo(data = dat, tau = t_list[j])
  reg_dat <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")
t_list <- c(2.5, 5, 7.5)
AX <- unique(dat[, c("idx", "A")])

reg_out <- data.frame(
  t_eval = t_list,
  beta_A = NA,
  p_A = NA
)

for (j in seq_along(t_list)) {
  t_eval <- t_list[j]
  pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
  reg_dat <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

  fit <- lm(pseudo ~ A, data = reg_dat)
  s <- summary(fit)$coefficients

  reg_out$beta_A[j] <- s["A", "Estimate"]
  reg_out$p_A[j] <- s["A", "Pr(>|t|)"]
}

reg_out
  fit <- lm(pseudo ~ A, data = reg_dat)
  s <- summary(fit)$coefficients

  reg_out$beta_A[j] <- s["A", "Estimate"]
  reg_out$p_A[j] <- s["A", "Pr(>|t|)"]
}

reg_out
```

```
##   t_eval     beta_A       p_A
## 1    2.5 -0.1275962 0.5905609
## 2    5.0 -0.4718782 0.5126343
## 3    7.5 -0.6096023 0.6652446
```

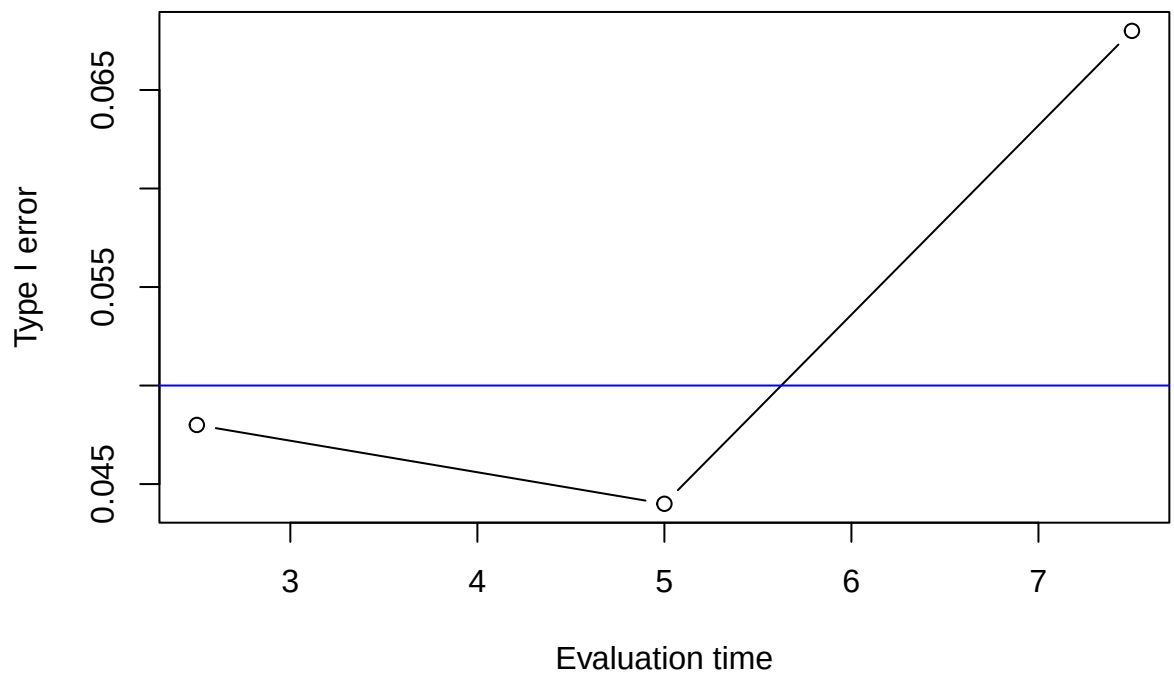
1.5 Type I error

```r
# Assisted by generative AI

set.seed(123)

B <- 500
alpha <- 0.05
t_list <- c(2.5, 5, 7.5)

type1_result <- data.frame(
  t_eval = t_list,
  reject_rate = NA,
  mean_beta = NA
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
  reject <- logical(B)
  beta <- numeric(B)

  for (b in 1:B) {

    source("01_generate_data.R")

    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)"] < alpha
  }

  type1_result$reject_rate[j] <- mean(reject)
  type1_result$mean_beta[j] <- mean(beta)
}

type1_result
```

```
##   t_eval reject_rate    mean_beta
## 1    2.5       0.048 -0.004524125
## 2    5.0       0.044 -0.002214517
## 3    7.5       0.068  0.019285168
```

```r
type1_result
```

```
##   t_eval reject_rate    mean_beta
## 1    2.5       0.048 -0.004524125
```

```
## 2    5.0       0.044 -0.002214517
## 3    7.5       0.068  0.019285168
```

```r
plot(type1_result$t_eval, type1_result$reject_rate, type = "b",
     xlab = "Evaluation time",
     ylab = "Type I error")
abline(h = 0.05, col = "blue")
```